Programming Test Results (With Test Cases)

Result Summary

Field	Value
Test ID	51989
Student ID	35631
Programs (with test cases)	5
Total Test Cases	18
Test Cases Passed	18
Fully Passed Programs	5
Partially Passed Programs	0
Failed Programs	0
Overall % (with test cases)	100.00%
Grade	Outstanding

Programs With Test Cases

#	Program Name	Total TC	Passed	Success Rate	Score /10	Submitted At	Attempts
1	AutomorphicNumberChecker	4	4	100.0%	10	13/10/2026, 12:21:18	0
2	StrongNumsInRange	4	4	100.0%	10	13/10/2026, 12:09:00	0
3	DuckNumberInRange	4	4	100.0%	10	13/10/2026, 11:54:33	0
4	HappyNumberChecker	3	3	100.0%	10	13/10/2026, 11:38:23	0
5	DisariumNumberChecker	3	3	100.0%	10	13/10/2026, 11:36:14	0

Program Details (With Test Cases)

Program 1: AutomorphicNumberChecker

Languages: Java

Score (010): 10 / 10

Test Case Summary: Total: 4 Passed: 4 Failed: 0 Success: 100.0%

Attempts: 0

Submitted At: 13/10/2026, 12:21:18

Description:

Automorphic Number Checker
==========

Problem Statement:
An Automorphic Number is a number whose square ends with the same digits as the original number.

Example 1: 25

25 = 625

The square 625 ends with 25.

Therefore, 25 is an Automorphic Number.

Example 2: 76

76 = 5776

The square 5776 ends with 76.

Therefore, 76 is an Automorphic Number.

Requirements

Write a Java program that:

Accepts a number from the user.
Calculates the square of the given number.
Checks whether the square ends with the original number.
If the square ends with the original number, display that the number is an Automorphic Number.

Otherwise, display that the number is not an Automorphic Number.

Examples of Automorphic Numbers
1, 5, 6, 25, 76, 376, 625, 9376, ...

Constraints: -

Sample Input: 25

Sample Output: 25 is an Automorphic Number

Explanation: -

Solution Code

```
import java.util.*;
class Automorphic{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        int n= sc.nextInt();

        int temp =n;
        int sq = temp*temp;

        while(n!=0){
            int r= sq%10;
            int r2 = n%10;
            if(r !=r2)
            {
                IO.println(temp +" is not an Automorphic Number");
                return;
            }
            sq = sq/10;
            n = n/10;
        }

        IO.println(temp +" is an Automorphic Number");


    }
}
```

Program 2: StrongNumsInRange

Languages: java

Score (010): 10 / 10

Test Case Summary: Total: 4 Passed: 4 Failed: 0 Success: 100.0%

Attempts: 0

Submitted At: 13/10/2026, 12:09:00

Description: Write a Java program to display all strong numbers from the given Limit.

Constraints: -

Sample Input: Enter the limit: 10

Sample Output: Strong numbers from 1 to 10 are: 1 2

Explanation: A Strong Number is a number whose sum of the factorials of its digits is equal to the number itself. Example: 145 1! + 4! + 5! = 1 + 24 + 120 = 145

Solution Code

```
import java.util.*;
class Strong{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        IO.println("Strong numbers from 1 to "+ n + " are:");
        for(int i=1;i<=n;i++){
            int sum =0;
            int temp =i;
            while(temp!=0){
                int p=1;
                int r = temp %10;
                for(int j=1; j<=r; j++){
                    p = p*j;
                }
                sum +=p;
                temp = temp/10;
            }
            if(sum ==i)IO.print(i + " ");
        }
    }
}
```

Program 3: DuckNumberInRange

Languages: Java

Score (010): 10 / 10

Test Case Summary: Total: 4 Passed: 4 Failed: 0 Success: 100.0%

Attempts: 0

Submitted At: 13/10/2026, 11:54:33

Description: Write a Java program to print all Duck Numbers in Range without using String conversion or comparison.

A Duck Number is a positive number that contains at least one digit 0, except that the number should not start with 0. Since integers never store leading zeros, simply check whether the number contains at least one digit 0.

Constraints: -

Sample Input: 1 100

Sample Output: 10 20 30 40 50 60 70 80 90 100

Explanation: -

Solution Code

```
import java.util.*;
class DuckNumber{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        int s = sc.nextInt();
        int l = sc.nextInt();

        if(s>l){
            IO.println("Invalid Range");
            return;
        }else if(s<0)
        {
            IO.println("Invalid Range");
            return;
        }

        for(int i=s;i<=l;i++){
            int n=i;
            while(n!=0){
                int r = n%10;
                n = n/10;
                if(r ==0) {
                    IO.print(i + " ");
```

```
                    break;
                }

            }
        }

    }
}
```

Program 4: HappyNumberChecker

Languages:	java	
Score (010):	10 / 10	
Test Case Summary:	Total: 3	Passed: 3
	Failed: 0	Success: 100.0%
Attempts:	0	
Submitted At:	13/10/2026, 11:38:23	

Description:

Happy Number Checker
==========

Write a Java program to check a given number is Happy number or Unhappy Number .

A happy number is a number that, when you replace it by the sum of the squares of its digits and repeat the process, eventually leads to the number 1.
If the process loops endlessly in a cycle that does not include 1, then the number is not a happy number.
However, it has been observed that all unhappy numbers end in the cycle 4 16 37 58 89 145 42 20 4.

So we have to stop the loop at 4.
Example 1 :
---
23 is a happy number because:

23 :> 2^2 + 3^2 = 13

13 :> 1^2 + 3^2 = 10

10 :> 1^2 + 0 = 1

As we reached to 1, 23 is a Happy Number.

Happy Numbers are : 1,7,10,13,19,23,28.....490,496

Example 2 :

---

42 is a UnHappy Number

42 :> 4^2 + 2^2 = 20

20 :> 2^2 + 0 = 4

As we reached to 4, 42 is a UnHappy Number.

Constraints: -

Sample Input: 19

Sample Output: Happy Number

Explanation: -

Solution Code

```
import java.util.*;
class HappyNumber{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int sum =0;
        while(n!=1 && n!=4){
            // sum of square of digits
            while(n!=0){
                int r =n%10;
                sum +=r*r;
                n = n/10;
            }
            n = sum;
            sum=0;
        }
        if(n==1)IO.println("Happy Number");
        else IO.println("UnHappy Number");

    }

}
```

Program 5: DisariumNumberChecker

Languages: Java

Score (010): 10 / 10

Test Case Summary: Total: 3 Passed: 3 Failed: 0 Success: 100.0%

Attempts: 0

Submitted At: 13/10/2026, 11:36:14

Description:

Disarium Number Checker
==========

A number is a Disarium number if the sum of its digits, each raised to the power of its respective position, is equal to the number itself.

For example:
---

89 is a Disarium number because :

89 = 8^1 + 9^2 = 8 + 81 = 89

175 is a Disarium number because :

175 = 1^1 + 7^2 + 5^3 = 1 + 49 + 125 = 175

45 is not a Disarium number because :

45 = 4^1 + 5^2 = 4 + 25 = 29 which is not equal to 45.

Some example of disarium numbers are : 1,89,135,175,518

Write a Java program that:
1. Accepts a number from the user.
2. Calculates the number of digits.
3. Raises each digit to the power of its respective position, starting from 1.
4. Calculates the sum of all these values.
5. Compares the calculated sum with the original number.
6. Displays whether the given number is a Disarium Number or Not a Disarium Number.

Constraints: -

Sample Input: 89

Sample Output: 89 is a Disarium Number

Explanation: -

Solution Code

```
import java.util.*;
class Disarium{
    public static void main(String [] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int temp =n;
        int l = (int)(Math.log10(n)) +1;

        int sum =0;
        while(temp !=0){
            int r=temp%10;
            int m = (int)(Math.pow(r,l));
            sum +=m;
            l--;
            temp = temp/10;
        }
        if(n==sum)IO.println(n +" is a Disarium Number");
        else IO.println(n + " is not a Disarium Number");
    }
}
```